Titanic — Machine Learning from Disaster

Random Forest workflow (parts A–F)

Project folder: `session_3/4.random_forest_titanic/` **Data:** `data/train.csv` (891 rows, 12 columns) and `data/test.csv` (418 rows) — the official Kaggle competition files. The same 891 training rows are also available without download via `seaborn.load_dataset("titanic")` (`survived`, `pclass`, `sex`, `age`, `sibsp`, `parch`, `fare`, `embarked`, …), so the notebook runs entirely from the libraries; the CSV files are used here because they also contain the 418-row test set needed for part F. **Code:** `src/titanic_rf.py` (one script, sections A–F, deterministic `random_state=42`) **Outputs:** `outputs/figures/`, `outputs/tables/`, `outputs/submission.csv`, `outputs/metrics_summary.json`

---

0. Setup: features, pipeline, and the leakage rule

Every model in this report is a scikit-learn `Pipeline`:

```
raw columns
  → row-wise features: Title, FamilySize, IsAlone, HasCabin
  → ColumnTransformer
       numeric     : SimpleImputer(median)
       categorical : SimpleImputer(mode) + OneHotEncoder(ignore unknown)
  → RandomForestClassifier / DecisionTreeClassifier
```

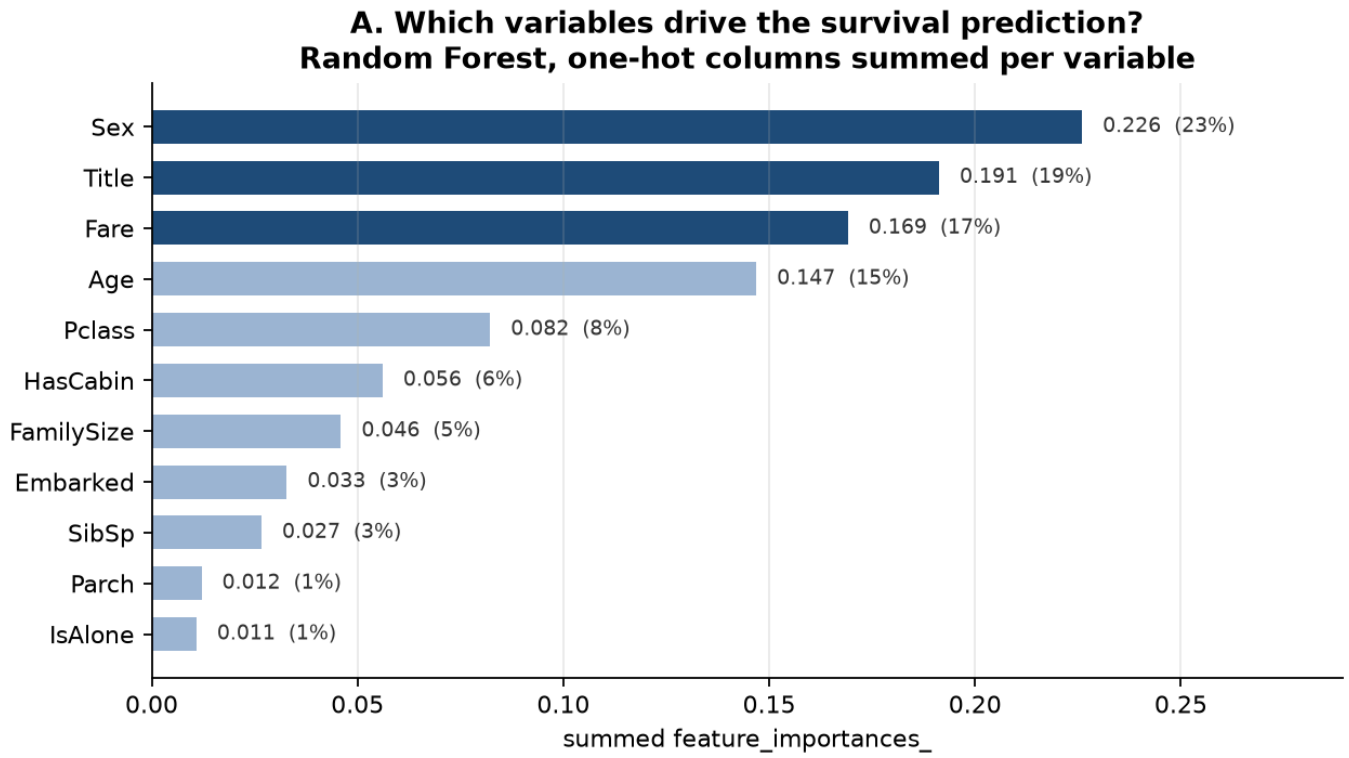
Feature	Type	Why it is there
`Sex`	categorical	the single strongest signal in 1912 (women and children first)
`Title` (from `Name`)	categorical	Mr / Mrs / Miss / Master / Officer / Noble / Rare — encodes sex, age and social role at once
`Pclass`	categorical	ticket class as a proxy for wealth and deck location
`Age`	numeric	children were prioritised; median-imputed (part B)
`Fare`	numeric	price paid, correlated with class
`SibSp`, `Parch`	numeric	family structure on board
`FamilySize`	numeric	`SibSp + Parch + 1`
`IsAlone`	categorical	`FamilySize == 1`
`HasCabin`	categorical	a recorded cabin means a better-located, more expensive ticket

Because the imputer sits **inside** the pipeline, it is fitted on training rows only and then applied to the validation/test rows. Nothing is learned from the data that is being predicted — this is what part B verifies explicitly.

Evaluation protocol. All "honest" numbers come from one stratified 20 % hold-out (`random_state=42`, 712 train / 179 validation rows). The final model in part F is then refitted on all 891 rows, as the hint suggests.

A. Feature importance from the Random Forest

`model.feature_importances_` returns one value per **encoded** column (so `Sex` becomes `Sex_female` + `Sex_male`). The chart below sums those columns back to their source variable, which is the readable version; the per-column detail is kept in `A_feature_importance_detail.png` and `tables/A_feature_importance.csv`.



Rank	Variable	Importance	Share
1	Sex	0.2260	22.6 %
2	Title	0.1913	19.1 %
3	Fare	0.1691	16.9 %
4	Age	0.1469	14.7 %
5	Pclass	0.0822	8.2 %
6	HasCabin	0.0561	5.6 %
7	FamilySize	0.0460	4.6 %
8	Embarked	0.0328	3.3 %
9	SibSp	0.0267	2.7 %
10	Parch	0.0121	1.2 %
11	IsAlone	0.0109	1.1 %

Reading the result. Sex and Title together carry 42 % of the total importance — and they overlap heavily, because `Title_Mr` is almost a copy of "is an adult man". Gender is the dominant rule; everything else refines it. `Fare` and `Age` come next: richer passengers and children had better odds. `SibSp` / `Parch` / `IsAlone` matter little *once* `FamilySize` is present, which is expected — the four variables describe the same thing.

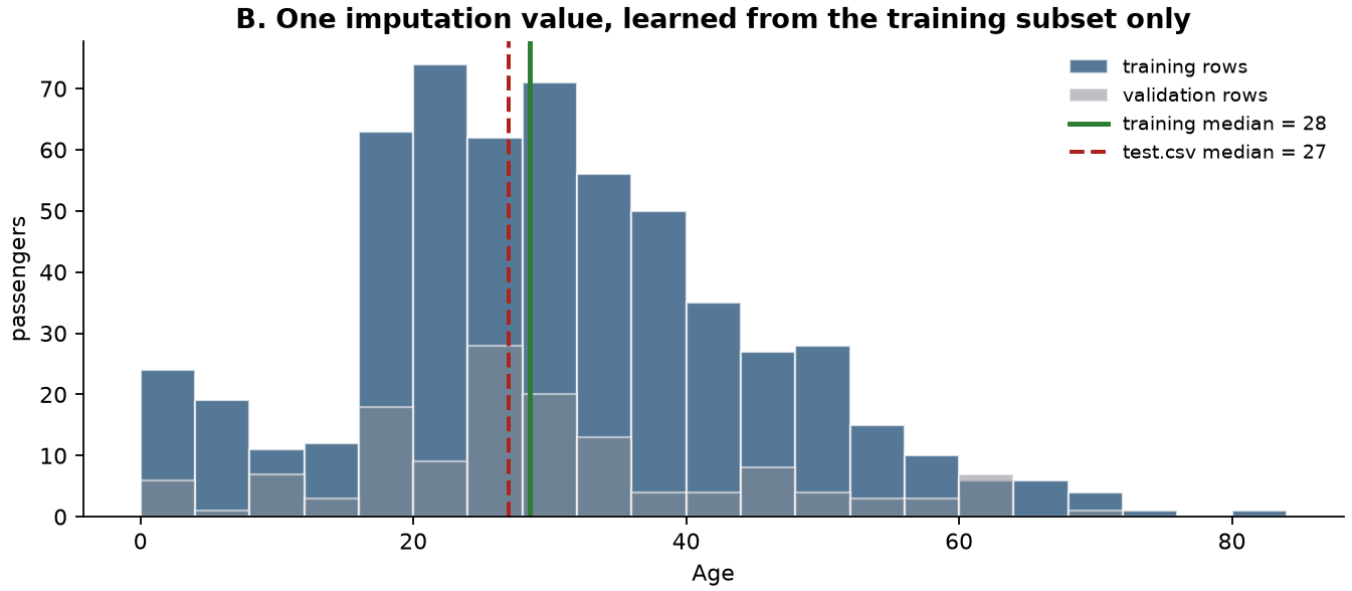
Caveat worth stating in any write-up: impurity-based importance is biased toward continuous, high-cardinality variables (here `Fare` and `Age`) and is computed on training data, so it measures "how much this variable was used to split", not "how much it causes survival".

B. Median age imputation learned from training rows only

The rule: compute the median on the **training subset**, store that single number, and use it to fill the missing ages in *both* subsets.

Subset	Rows	Missing `Age`	Median `Age`
Training rows (80 % split)	712	137	**28.5**
Validation rows (20 % split)	179	40	27.0
Kaggle `test.csv`	418	86	27.0
All 891 training rows	891	177	28.0
Train + test pooled (leaky)	1309	263	28.0

The three subsets do not share the same median (28.5 / 27.0 / 27.0). Filling everything with one of the "wrong" numbers would quietly move 177 imputed passengers by 1–1.5 years — small here, but the same mistake on a strongly skewed column (e.g. `Fare`) is not small at all.



Two implementations, one result. Filling `Age` manually with `median_train = 28.5` and letting `SimpleImputer(strategy="median")` learn the value from `X_train` give the *identical* validation predictions (`identical predictions: True`), and the imputer's learned statistic is exactly `28.5`. That is the practical proof that the pipeline is leak-free.

Does the leak actually hurt? Measured on the same hold-out:

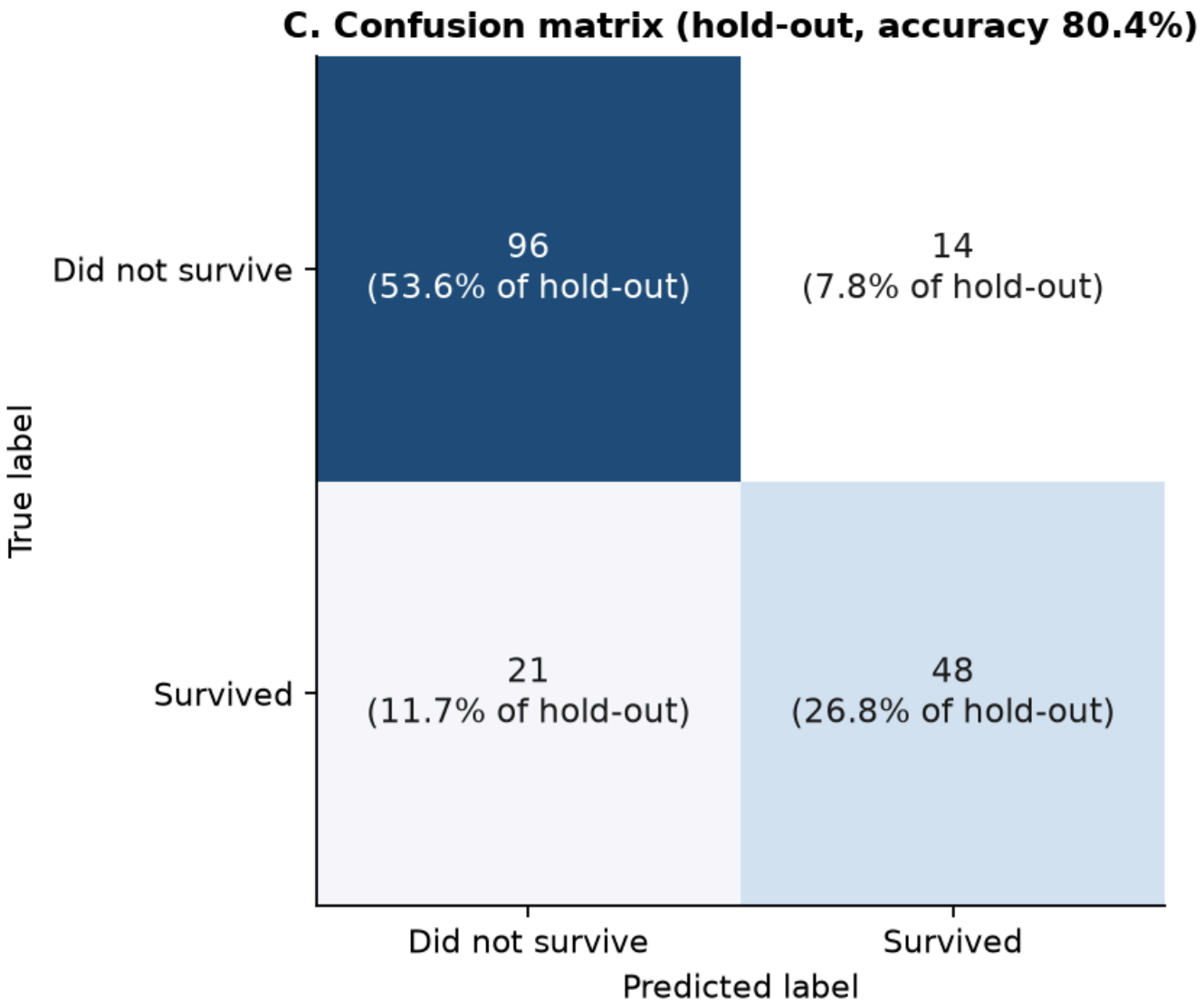
Imputation value taken from	Value used	Rows whose age changes	Validation accuracy
Training rows (correct)	28.5	0	**0.8045**
Validation rows (leak)	27.0	177	0.8045
`test.csv` rows (leak)	27.0	177	0.8045
Train + test pooled (leak)	28.0	177	0.7989

So on this split the leak changes accuracy by 0 to −0.6 points: a tree that only asks "is this passenger older than ~6 or ~30?" barely notices a one-year shift. The lesson is not "leakage is harmless" — it is that a single hold-out of 179 rows is too coarse to detect it, while the *practice* of learning imputation values from training rows only is free and always correct.

C. Classification report and confusion matrix

Evaluated on the 179 held-out passengers (positive class = **Survived**):

	precision	recall	f1-score	support
Did not survive	0.8205	0.8727	0.8458	110
Survived	0.7742	0.6957	0.7328	69
accuracy			**0.8045**	179
macro avg	0.7974	0.7842	0.7893	179
weighted avg	0.8027	0.8045	0.8023	179



	Predicted **Did not survive**	Predicted **Survived**
Actual Did not survive	96 (TN)	14 (FP)
Actual Survived	21 (FN)	48 (TP)

144 of 179 passengers are classified correctly (80.4 %), versus 61.5 % for the trivial “everybody dies” baseline.

D. False positives and false negatives

With **Survived** as the positive class:

- **False positive (Type I, 14 passengers, 7.8 % of the hold-out)** — the model predicted *Survived*, the passenger died. These are almost all **women travelling alone in 3rd class** (`Miss`, low fare, no family): the model has learned “female → survives” very strongly and has no feature that can tell it this particular woman was below deck when the ship went down. A few confident men (e.g. a 1st-class doctor with a 90-fare ticket) also slip in through the “wealthy → survives” pattern.
- **False negative (Type II, 21 passengers, 11.7 % of the hold-out)** — the model predicted *Did not survive*, the passenger survived. These are almost all **men in 2nd/3rd class with a cheap ticket and no family**, the group with the lowest survival rate (~14 %). The model assigns them a probability near 0.00–0.06, so it is not “unsure” — it is applying a rule that is right four times out of five.

	Precision	Recall	F1	Support
Did not survive	0.8205	**0.8727**	0.8458	110
Survived	0.7742	0.6957	0.7328	69

Which class has the higher recall? *Did not survive* (0.873 vs 0.696). The model catches 87 % of the passengers who died but only 70 % of those who lived: it is biased toward the majority class (61.6 % of the training data did not survive), and the female-survivor rule it relies on is not strong enough to recover the remaining men who survived.

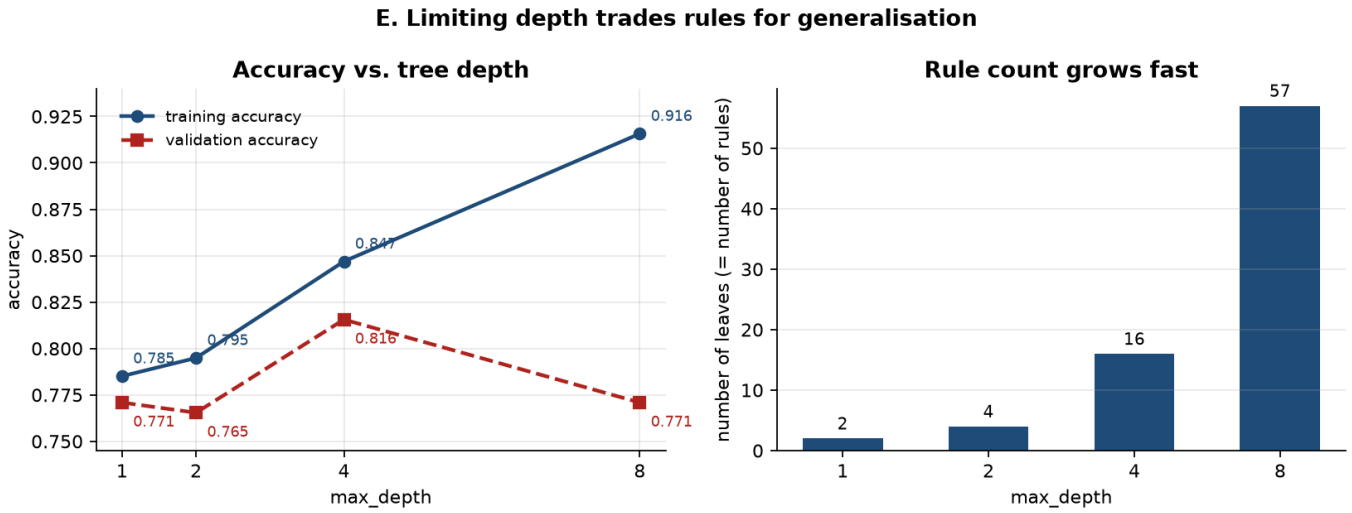
Cost asymmetry. The two error types are not equally expensive in the real world. A false negative (“this passenger will die”, but they live) is a missed rescue opportunity; a false positive (“will survive”) sends resources to someone who does not need them. If recall on survivors mattered more, the levers are: lower the decision threshold (`predict_proba(...)[:, 1] >`

`0.4`), use `class_weight="balanced"`, or optimise for F1/recall on the positive class instead of plain accuracy. Each of those trades accuracy for survivor recall — a choice the competition metric (plain accuracy) does not reward, which is why the default 0.5 threshold is kept here.

E. Four decision trees: `max_depth` = 1, 2, 4, 8

Each tree is fitted on the same 712 training rows with `random_state=42`; "actual depth" is `tree.get_depth()` and a leaf is one rule.

max_depth	actual tree depth	leaves (= rules)	nodes	training accuracy	validation accuracy	train − val gap
1	1	2	3	0.7851	0.7709	0.014
2	2	4	7	0.7949	0.7654	0.030
4	4	16	31	0.8469	**0.8156**	0.031
8	8	57	113	**0.9157**	0.7709	**0.145**



The rules, and how they change.

Depth 1 — one question, two rules.

```
Title_Mr ⩽ 0.5  →  Survived      (300 passengers, 70 % survived)
Title_Mr >  0.5  →  Did not survive (412 passengers, 15 % survived)
```

Interesting detail: the tree does **not** split on `Sex`, it splits on `Title_Mr`. `Title_Mr` is a *better* version of "male", because it excludes `Master` (boys), who mostly survived. So even the shallowest tree already encodes "adult man ⇒ dies, otherwise ⇒ lives".

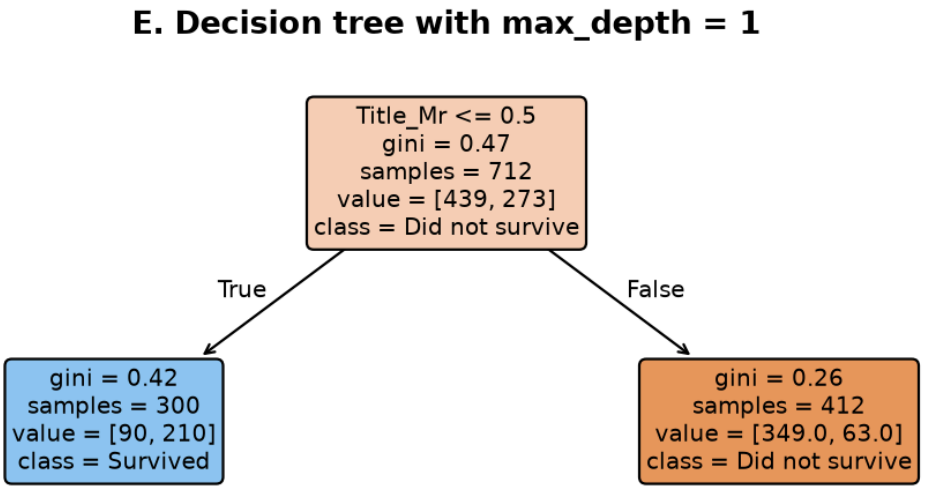
Depth 2 — four rules, the first interaction.

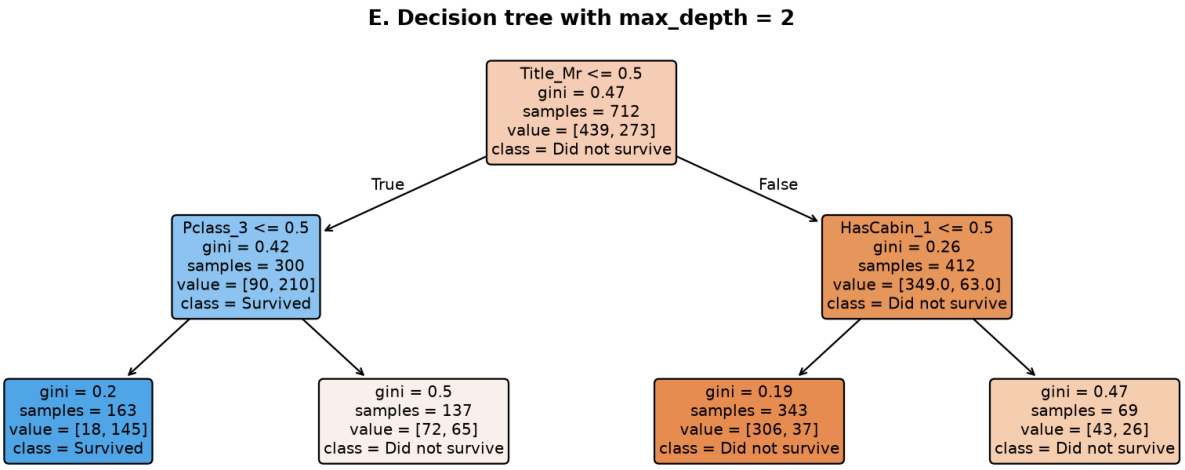
```
Title_Mr ⩽ 0.5 → Pclass_3 ⩽ 0.5 → Survived          (163, 89 % survived)
               → Pclass_3 > 0.5 → Did not survive   (137, 47 % survived)
Title_Mr > 0.5 → HasCabin_1 ⩽ 0.5 → Did not survive (343, 11 % survived)
               → HasCabin_1 > 0.5 → Did not survive  (69, 38 % survived)
```

The second split is now *conditional*: for non-Mr passengers the tree asks about class (3rd-class women and children were much worse off), for Mr passengers it asks about cabin. That is exactly what depth buys — a rule that only applies inside a subgroup.

Depth 4 — 16 rules. The tree starts using `Age` (children), `Fare` and `FamilySize`; accuracy jumps to 0.847 on training and 0.816 on validation.

Depth 8 — 57 rules. Training accuracy reaches 0.916 while validation *falls back* to 0.771, below the depth-4 tree. The extra 41 rules describe individual passengers (a specific fare, a specific family), not a pattern: pure memorisation.





Conclusion for part E. Depth is a complexity dial with an optimum in the middle. Going 1 → 2 → 4 adds genuine interactions and improves generalisation; going 4 → 8 multiplies the rule count by 3.5 and only buys training accuracy. The train–validation gap grows from 1.4 points (depth 1) to 14.5 points (depth 8) — the classic overfitting signature. A single deep tree is also unstable: a different `random_state` or a different 20 % split would give a visibly different tree. That instability is precisely what the 500-tree forest in part F fixes.

F. Final model and the Kaggle submission

Final configuration — chosen with a 5-fold CV grid over `min_samples_leaf` in {1, 2, 3}, `max_features` in {`sqrt`, 0.4} and `class_weight` in {None, `balanced_subsample`} (12 candidates):

```
RandomForestClassifier(n_estimators=500,
                       min_samples_leaf=2,
                       max_features=0.4,
                       random_state=42, n_jobs=-1)
```

`min_samples_leaf=2` and `max_features=0.4` both smooth the individual trees — fewer one-passenger leaves, more feature diversity per split.

Model	5-fold CV accuracy
Sex rule (single split, depth-1 equivalent)	0.7767
Decision Tree, `max_depth=4`	0.8126
Random Forest, 500 trees (defaults)	0.8081
Random Forest, 500 trees (tuned)	**0.8361**

Following the hint, the tuned forest is then **refitted on all 891 training rows** and used to predict the 418 test passengers.

Accuracy estimate for the test set.

Estimate	Value
5-fold CV on the 891 training rows	0.8361 ± 0.0127
5×5 repeated stratified CV (25 fits)	**0.8294 ± 0.0200**
Single 20 % hold-out	0.8045

The repeated-CV figure (0.829) is the number to quote: a single 5-fold split is itself a noisy estimate, and the 179-row hold-out moves by ±3 points from one split to the next. On the 418-row Kaggle test set, expect roughly 0.77–0.80 on the public leaderboard — the test passengers are a different sample, and leaderboard noise on 418 rows is ±2 points.

Submission file — `outputs/submission.csv`, exactly the `gender_submission.csv` shape:

```
PassengerId,Survived
892,0
893,0
894,0
...
```

Format check	Result
Columns `PassengerId`, `Survived`	pass
418 rows + header	pass
`PassengerId` identical to the sample file	pass
No missing values	pass
Only 0/1 values	pass
Predicted survival rate	0.371 (training rate: 0.384)

To submit: upload `outputs/submission.csv` on the *Titanic — Machine Learning from Disaster* competition page → *Submit Predictions* → the leaderboard shows your public score immediately (or `kaggle competitions submit -c titanic -f submission.csv -m "RF tuned"` with the Kaggle API configured).

Summary of what the six parts show

1. **Sex/Title dominate** (42 % of importance), then Fare, Age and Pclass — the historical "women, children and first class first" pattern is what the model actually learned.
2. **Imputation must be learned on training rows only**; the pipeline guarantees it, and on this data the effect on accuracy is small (0 to −0.6 pt) but the practice costs nothing.
3. **80.4 % hold-out accuracy**, with 87 % recall on the deceased and 70 % on survivors.
4. **Errors are structured, not random**: false positives are lone 3rd-class women, false negatives are cheap-ticket men — exactly the groups whose outcome the available features cannot resolve.
5. **Depth 4 is the sweet spot for a single tree**; depth 8 memorises (train 0.916 / validation 0.771). The forest removes that variance and lifts CV accuracy to **0.836**.
6. **Submission ready**: 418 predictions, format-verified, trained on all 891 rows as recommended.

Reproducing

```
cd session_3/4.random_forest_titanic
python src/titanic_rf.py   # figures, tables, submission.csv
```

Deterministic: `random_state=42` everywhere; the 20 % split is stratified.